

FitBuddy AI – Intelligent Full-Stack Adaptive Fitness & Nutrition Coach

Project Description:

FitBuddy AI is a full-stack, AI-powered fitness and nutrition assistance platform that generates personalized workout plans and nutrition or recovery guidance from user-provided fitness information. It is built as a decoupled web application: a Streamlit frontend running at <http://localhost:8501> communicates with a FastAPI backend served by Uvicorn at <http://127.0.0.1:8000>, and the AI content is generated with Google Gemini through the Google Generative AI interface.

The application accepts user information such as name, age, weight, fitness goal, workout intensity and other available fitness metrics. From this information it produces a personalized 7-day workout plan, a nutrition or recovery tip and relevant fitness information. Users can also give feedback, and the system produces an adaptive updated plan based on that feedback.

FitBuddy AI is designed as a complete application rather than only an AI prompt. Each request passes through input processing (including BMI calculation where applicable), Gemini generation with retry handling, and a deterministic fallback if the AI service cannot respond. The result is stored in a SQLite database using SQLAlchemy and returned to the Streamlit interface, where the user sees the personalized plan.

Scenarios:

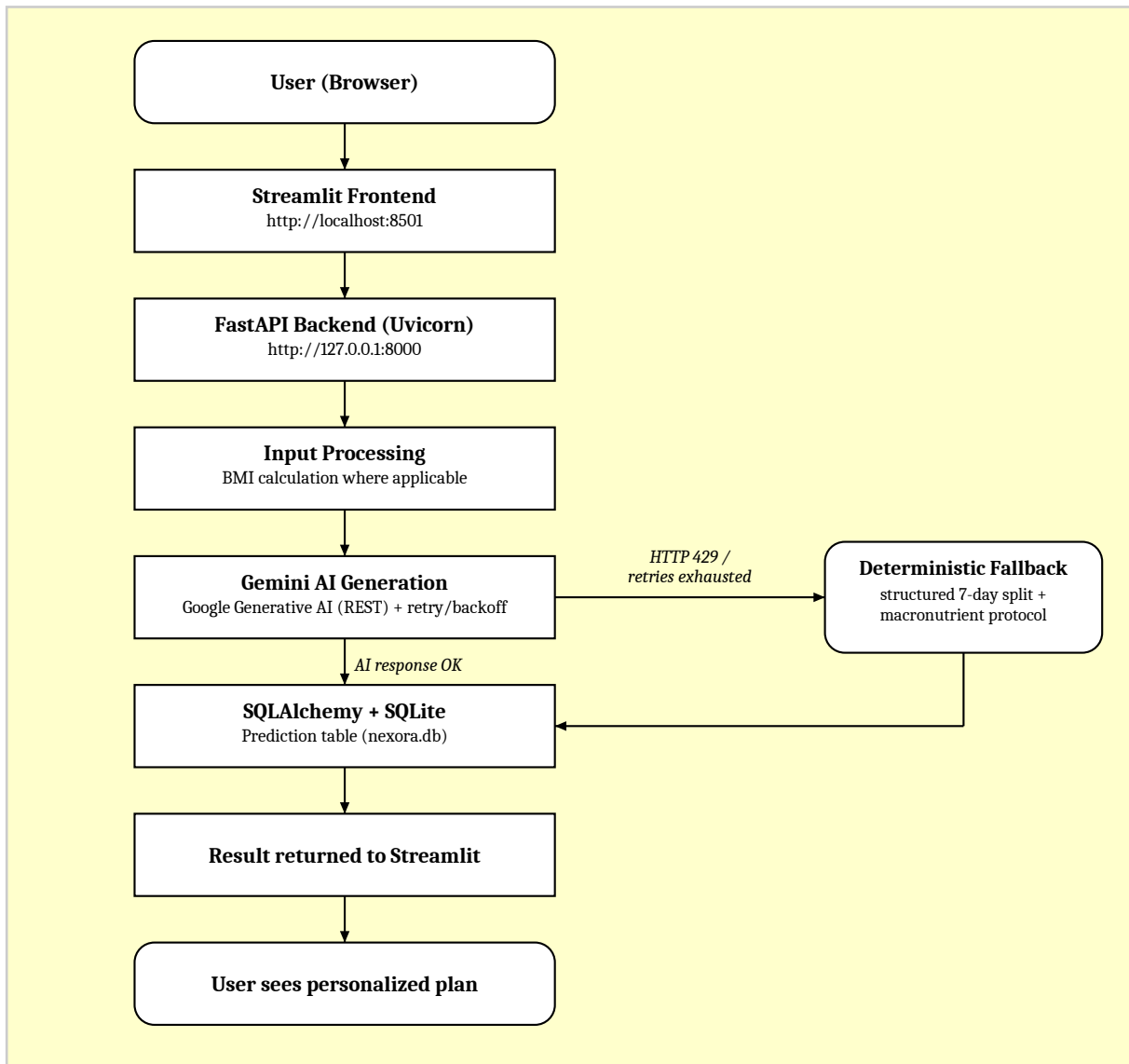
Scenario 1 : A user enters their name, age, weight, fitness goal (e.g., “weight loss” or “muscle gain”), workout intensity and other available fitness metrics in the Streamlit interface. The FastAPI backend processes the input, calculates BMI where applicable and requests a personalized 7-day workout plan from Gemini. The generated plan is stored in the database and displayed to the user as a structured, day-by-day schedule.

Scenario 2: A user submits feedback on the existing plan, such as asking for “more focus on cardio” or “include more rest days.” The system uses this feedback to produce an adaptive updated plan, so that the plan keeps matching the user’s changing needs and preferences.

Scenario 3: A user receives a nutrition or recovery tip related to the selected fitness goal. This guidance complements the workout plan by helping the user align eating and recovery habits with the training routine.

Scenario 4: While a plan is being generated, the Google Gemini free tier returns an HTTP 429 rate-limit error. The backend retries the request with backoff delays. If the AI request still cannot be completed, the system does not crash the user interface; it returns a deterministic, structured 7-day progressive workout split with a macronutrient-oriented protocol. This keeps the application functional and is a software reliability mechanism.

Technical Architecture:



Pre - requisites:

- 1 FastAPI Framework Knowledge: [FastAPI Documentation](#)
- 2 Gemini API Familiarity: [Google Generative AI Documentation](#)
- 3 Streamlit Basics: [Streamlit Documentation](#)
- 4 Python Programming Proficiency: [Python Official Docs](#)
- 5 Version Control with Git and GitHub: [Git Documentation](#)
- 6 SQLAlchemy and SQLite Basics: [SQLAlchemy Docs](#)
- 7 Environment Setup (pip, virtualenv): [Virtualenv Guide](#)
- 8 Uvicorn ASGI Server: [Uvicorn Docs](#)

Project Workflow:

Milestone 1. Model selection and Architecture

Activity 1.1. Research and Select the Appropriate Generative AI Model

Activity 1.2. Define the architecture of the application

Activity 1.3. Set up the development Environment

Milestone 2. Core functionalities Development

Activity 2.1. Develop the core functionalities

Activity 2.2. Implement the FastAPI Backend and Database Persistence

Milestone 3. Gemini Integration, Retry Handling and Deterministic Fallback

Activity 3.1. Integrating Google Gemini

Activity 3.2. Retry, Backoff and HTTP 429 Handling

Activity 3.3. Implementing the Deterministic Fallback

Milestone 4. Frontend Development

Activity 4.1. Designing and Developing the Streamlit User Interface

Activity 4.2. Frontend-Backend Integration

Milestone 5. Deployment and Testing

Activity 5.1. Preparing the Application for Local Deployment

Activity 5.2. Testing and Verifying the Application

Results, Limitations, Future Enhancements, Skill Wallet and Conclusion

Milestone 1. Model selection and Architecture

In this milestone, we focus on selecting the generative AI model for the FitBuddy AI application and defining how the frontend, backend, AI service and database work together. The model must support generating personalized workout plans, updating them based on feedback, and providing nutrition or recovery guidance.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

FitBuddy AI's AI model needed to:

- Generate structured, personalized 7-day workout plans
- Provide concise and actionable nutrition or recovery guidance
- Support adaptive plan updates based on user feedback
- Be accessible through an API from a Python backend, without local GPU hosting

Final Model Selection

The project uses **Google Gemini**, integrated through the Google Generative AI interface with REST transport. The application uses the Gemini model configured in the application settings, so the model can be changed in configuration without redesigning the system. Two generation modules use Gemini:

- `generate_workout_gemini()` – generates the personalized 7-day workout plan

- `generate_nutrition_tip_with_flash()` – generates the nutrition or recovery tip; the function name reflects its role as the fast, lightweight generation call

Reasons for Choosing Gemini:

- API-based and easy to integrate with a FastAPI backend
- Produces structured, day-wise text (days, sections) from clear prompts
- No local model hosting or GPU setup required
- A free tier is available for student development; its rate limits are handled in Milestone 3

Activity 1.2. Define the architecture of the application

Architecture Overview: FitBuddy AI follows a **decoupled full-stack architecture**, where the user interface and the server logic run as separate applications and communicate over HTTP.

- Frontend: Streamlit application (<http://localhost:8501>) for user input and display of results
- Backend: FastAPI application served by Uvicorn (<http://127.0.0.1:8000>) for request handling and logic
- AI Layer: Google Gemini through the Google Generative AI interface (REST transport)
- Database: SQLite with SQLAlchemy ORM for persistence

Main workflow of the application:

```
User Input
-> Streamlit Frontend
-> FastAPI Backend
-> Input Processing / BMI Calculation (where applicable)
-> Gemini AI Generation
-> Retry Handling
-> Deterministic Fallback (if required)
-> SQLite Database
-> Result returned to Streamlit
-> User sees personalized plan
```

Frontend Responsibilities:

- Collects name, age, weight, fitness goal, workout intensity and other available fitness metrics
- Sends the user data to the FastAPI backend
- Displays the 7-day workout plan, the nutrition or recovery tip and the success indication

Backend Responsibilities:

- Receives and processes requests from the frontend
- Calculates BMI where applicable
- Calls the Gemini generation modules with retry and backoff handling
- Uses the deterministic fallback when the AI request cannot be completed
- Stores results in the SQLite database through SQLAlchemy and returns them to the frontend

AI Integration Points:

- Workout plan generation and adaptive plan updating → Gemini
- Nutrition or recovery guidance → Gemini

Activity 1.3 Set up the development Environment:

Component	Details
Operating System	Windows 11
Python	Python 3.10.11
Virtual Environment	fitbuddy-env
Code Editor	Visual Studio Code
Version Control	Git and GitHub (main branch synchronized)
Backend URL	http://127.0.0.1:8000
Frontend URL	http://localhost:8501
Project Directory	C:\Users\sriha\OneDrive\Documents\GitHub\sanjay-fit

Install Python & Pip: Ensure Python and pip are installed.

Create Virtual Environment:

```
python -m venv fitbuddy-env
fitbuddy-env\Scripts\activate (Windows)
```

Install Required Libraries:

```
pip install fastapi uvicorn streamlit sqlalchemy google-generativeai
```

Run the Backend Server:

```
uvicorn main:app --reload
```

Visit <http://127.0.0.1:8000> to confirm the backend is running and [/docs](#) for interactive API testing. The Streamlit frontend is started with the `streamlit run` command followed by the frontend script name and opens at <http://localhost:8501>.

Set Up Project Structure: The main files discussed in this report are organized as follows.

```
sanjay-fit/
|
|-- main.py           # FastAPI application, routes, table creation
|-- database.py       # SQLite URL, SQLAlchemy engine, SessionLocal, Base
|-- models.py         # Prediction table (SQLAlchemy model)
|-- Gemini generation module # workout + nutrition functions, retry/backoff, fallback
|-- Streamlit frontend script # user interface at http://localhost:8501
|-- nexora.db         # Local SQLite database file (auto-generated)
```

Milestone 2. Core functionalities Development

Activity 2.1 Develop the core functionalities:

Workout Plan Generation

Function: `generate_workout_gemini()`

Alias: `generate_workout_plan`

This function implements the core logic to generate a personalized 7-day workout plan. It accepts flexible parameters – name, age, weight, goal, intensity and additional keyword arguments – builds a prompt from the user's fitness information and sends it to Gemini, which returns a structured, day-wise plan. The output is stored and returned to the Streamlit frontend for display.

```
def generate_workout_gemini(name, age, weight, goal, intensity, **kwargs):
    # build prompt from user inputs -> call Gemini (with retry/backoff)
    # -> return structured 7-day plan (or deterministic fallback)
    ...

generate_workout_plan = generate_workout_gemini    # alias
```

Nutrition / Recovery Tip Generation

Function: `generate_nutrition_tip_with_flash()`

Alias: `generate_nutrition_tip`

This function generates a concise nutrition or recovery tip related to the user's fitness goal and inputs. The tip complements the workout plan and is displayed together with it in the Streamlit interface.

```
generate_nutrition_tip = generate_nutrition_tip_with_flash    # alias
```

Implementation Fix 1: Modular Function Mapping

During integration, ImportError and function-name mismatch problems occurred because different parts of the project referred to the generation functions by different names. These were solved by keeping the main functions and adding aliases:

Main Function	Alias	Purpose
generate_workout_gemini	generate_workout_plan	7-day workout plan generation
generate_nutrition_tip_with_flash	generate_nutrition_tip	Nutrition / recovery tip generation

Implementation Fix 2: Flexible Parameter Handling

The generation functions accept dynamic keyword arguments such as name, age, weight, goal, intensity and ****kwargs**. This prevents unexpected-argument errors when the data sent by the frontend or the backend integration changes, and allows additional fitness metrics to be passed without rewriting the function signature.

Input Processing and BMI Calculation

Before the AI request is made, the backend processes the received user data. Where the required measurements are available, the Body Mass Index (BMI) is calculated as weight in kilograms divided by the square of height in metres ($BMI = \text{weight} / \text{height}^2$) and can be used as additional context for plan generation and storage.

Feedback-Based Adaptive Planning

FitBuddy AI supports adaptive planning. After viewing a plan, the user can give feedback (for example, “more focus on cardio” or “include more rest days”). The feedback is combined with the user’s existing plan information and submitted for regeneration, and the adaptive updated plan is returned to the frontend. This feedback loop keeps the plan aligned with the user’s progress and preferences instead of leaving it fixed after the first generation.

User & Plan Storage – Database and Persistence

File: **database.py**, **models.py**

The application stores its data in a SQLite database accessed through the SQLAlchemy ORM. The database configuration in database.py is:

```
SQLALCHEMY_DATABASE_URL = "sqlite:///./nexora.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()
```

The setting **check_same_thread=False** allows SQLite to be used safely from FastAPI’s request handling threads. **SessionLocal** creates database sessions for each request, and **Base** is the declarative base class for all models.

Current Prediction table (models.py):

Field	Description
id	Unique record identifier
location	Location associated with the record
risk_type	Type of risk / category of the record
risk_level	Level assigned to the record
confidence	Confidence value of the result
result	Generated result stored for the record
created_at	Timestamp of record creation

Important – current implementation vs. project concept: The fields above are the fields of the Prediction model that exist in the current backend. The broader FitBuddy AI concept of storing user metrics, calculated BMI and complete generated workout/nutrition plans is the intended use of the persistence layer; user-specific fields such as age, weight or BMI are not yet separate columns of the current database model. Extending the schema is listed under Future Enhancements.

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

The FastAPI application is defined in `main.py`. It creates the database tables at startup and exposes the following endpoints:

Endpoint	Method	Description
/	GET	Returns a confirmation that the backend is running
/predictions	POST	Stores a prediction / result record in the SQLite database
/predictions	GET	Retrieves the stored records from the database

Process User Input

- The Streamlit frontend collects the user's name, age, weight, goal, intensity and other available metrics
- The data is sent to the FastAPI backend over HTTP
- The backend processes the input, calls the generation functions and saves the result with SQLAlchemy sessions
- The generated result is returned to the frontend as the response

Integrate Gemini API Calls

The route logic connects directly to the Gemini-powered functions `generate_workout_gemini()` and `generate_nutrition_tip_with_flash()`. All AI responses (or fallback results) are returned cleanly to the frontend, which renders them for the user.

Milestone 3. Gemini Integration, Retry Handling and Deterministic Fallback

Milestone 3 focuses on the reliability of the AI layer. Because the application depends on an external AI service with free-tier limits, the project includes retry logic, rate-limit handling and a deterministic fallback so that the user interface remains usable at all times.

Activity 3.1: Integrating Google Gemini

- Gemini is accessed through the Google Generative AI interface using REST transport
- The Gemini model is taken from the application configuration, so it can be changed without redesigning the system
- Prompts are built from the user inputs (goal, intensity, age, weight and other metrics) and, for adaptive updates, from the user's feedback
- The API key should be kept in an environment variable or a .env file and must not be committed to GitHub

Activity 3.2: Retry, Backoff and HTTP 429 Handling

The Google Gemini free tier limits the number of requests that can be made in a period of time. When the limit is exceeded, the service returns an **HTTP 429 (Too Many Requests)** error. If this error were passed directly to the interface, the application would fail for the user. FitBuddy AI therefore includes:

- **Retry logic** – a failed Gemini request is attempted again instead of failing immediately
- **Backoff delays** – the application waits between attempts, giving the rate limit time to recover
- **Graceful fallback** – if the AI request still cannot be completed, a deterministic structured result is returned and the UI does not crash

Situation	System behaviour
Gemini responds successfully	The Gemini-generated plan / tip is used, stored and displayed
Gemini returns HTTP 429	The request is retried after a backoff delay
Request cannot be completed after retries	The deterministic fallback result is returned; the frontend does not crash

Activity 3.3: Implementing the Deterministic Fallback

The deterministic fallback keeps the application functional even when Gemini is temporarily unavailable. It returns a fixed, rule-based structure and therefore does not depend on any external service. It provides:

- A structured 7-day progressive workout split
- A macronutrient-oriented nutrition protocol

The purpose of the fallback is to guarantee that the user always receives a structured response and that the pipeline (generate → save → display) always completes.

Note: The deterministic fallback is a **software reliability / fault-tolerance mechanism**. It is not a medically validated personalized prescription. Both AI-generated and fallback results are general fitness information, and users with medical conditions should consult a qualified professional before starting a new exercise or diet plan.

Milestone 4. Frontend Development

Milestone 4 focuses on the user interface of FitBuddy AI. The frontend is built with **Streamlit**, a Python framework for interactive web applications, and runs at `http://localhost:8501`. It is separate from the FastAPI backend and communicates with it over HTTP.

Activity 4.1 Designing and Developing the Streamlit User Interface

User Input Section

- Collects the user's name, age, weight, fitness goal and workout intensity (for example low, medium or high)
- Accepts other available fitness metrics where provided
- A submit action sends the entered data to the backend

Result Section

- Displays the personalized 7-day workout plan
- Displays the nutrition or recovery tip and relevant fitness information
- Shows a success indication after the result has been generated and saved
- Provides the option to give feedback and view the adaptive updated plan

Activity 4.2 Frontend-Backend Integration

- Streamlit sends the user data to the FastAPI backend at `http://127.0.0.1:8000`
- The backend processes the request, obtains the Gemini or fallback result and stores it in SQLite
- The response is returned to Streamlit and rendered for the user
- Flexible keyword-argument handling in the generation functions keeps the integration stable when the data exchanged between frontend and backend changes

Exploring the application's pages:

Home page (user input):

[Paste screenshot here]
Streamlit input page – <http://localhost:8501>

Description: The home page of FitBuddy AI presents the input form where the user enters the name, age, weight, fitness goal, workout intensity and other available metrics. Submitting the form starts the pipeline: the data is sent to the FastAPI backend, which generates the plan and returns it to the interface.

Personalized workout plan page:

[Paste screenshot here]
7-day workout plan displayed in Streamlit

Description: After the backend returns the result, the interface displays the personalized 7-day workout plan day by day, based on the user's goal and intensity. The plan comes from Gemini or, when the AI service cannot respond, from the deterministic fallback, so a structured plan is always shown.

Nutrition / Recovery Tip:

[Paste screenshot here]
Nutrition or recovery tip section

Description: The nutrition or recovery section provides concise, practical guidance that complements the workout plan and the selected fitness goal. It helps users connect their training with eating and recovery

habits.

Feedback and Adaptive Plan:

[Paste screenshot here]
Feedback input and updated plan

Description: The feedback section lets the user comment on the plan. The feedback is used to produce an adaptive updated plan, which is shown to the user so that the plan evolves with their needs.

Stored Results (FastAPI /docs or GET /predictions):

[Paste screenshot here]
Stored records retrieved from the SQLite database

Description: Stored records can be retrieved from the backend through the GET /predictions endpoint, which can also be tested in the interactive FastAPI documentation at <http://127.0.0.1:8000/docs>.

Milestone 5. Deployment and Testing

This milestone covers running FitBuddy AI on a local system, verifying the complete pipeline, and managing the project with Git and GitHub. The application currently runs locally, and the decoupled design prepares it for future cloud deployment.

Activity 5.1: Preparing the Application for Local Deployment

Set up the virtual environment

- Create and activate the virtual environment `fitbuddy-env` to isolate project dependencies
- Install FastAPI, Uvicorn, Streamlit, SQLAlchemy and the Google Generative AI library

Configure environment variables

- Store the Gemini API key in an environment variable or `.env` file instead of in the source code

Start the application

Step	Action	URL
1	Start the FastAPI backend with Uvicorn (<code>uvicorn main:app --reload</code>)	<code>http://127.0.0.1:8000</code>
2	Start the Streamlit frontend with the <code>streamlit run</code> command	<code>http://localhost:8501</code>
3	Open the Streamlit page in a browser, enter details and generate a plan	<code>http://localhost:8501</code>
4	Optionally test the API endpoints in the interactive documentation	<code>http://127.0.0.1:8000/docs</code>

Git and GitHub

- A Git repository was created for the project and is synchronized with GitHub
- The main branch is up to date
- Version control allowed the two team members to track changes and keep their work consistent

Activity 5.2: Testing and Verifying the Application

The project was tested end-to-end. Each stage of the main pipeline was checked, and the pipeline is working:

Stage	Component	Verification
1	Streamlit UI sends the user data	Verified
2	FastAPI receives and processes the request	Verified
3	Gemini or the fallback generates the result	Verified
4	Result saved to SQLite database	Verified
5	Result displayed in the frontend	Verified
6	Success indication shown to the user	Verified

The backend endpoints GET /, POST /predictions and GET /predictions provide a direct way to check that the server is running and that records are stored and retrieved.

Results and Discussion:

- FitBuddy AI works as a complete full-stack application: Streamlit input leads to a stored result displayed back in the interface.
- The decoupled Streamlit–FastAPI design lets the user interface and the server logic be developed and run separately.
- Function aliases and flexible parameters solved the integration problems between modules.
- Retry with backoff and the deterministic fallback prevent HTTP 429 errors from breaking the user experience.
- Results are persisted using SQLAlchemy and SQLite, and the project is maintained with Git and GitHub.

The main trade-off in the design is between personalization and availability. Gemini-generated plans are produced from the user's inputs, whereas the deterministic fallback is a fixed progressive structure; it is less individualized but guarantees that the application keeps working.

Limitations:

- The current database model (Prediction table) does not yet contain separate columns for user metrics such as age, weight or BMI, or for full workout and nutrition plans.
- The deterministic fallback is a general, rule-based structure and is not a medically validated personalized prescription.
- Gemini free-tier rate limits can delay or prevent AI generation; the fallback only keeps the application functional.
- The application is currently deployed only on a local system (Windows 11).
- Generated plans are general fitness information and do not replace advice from qualified fitness or medical professionals.
- User login and authentication are outside the documented scope of the current version.

Future Enhancements:

- Extend the SQLAlchemy models to store complete user profiles, calculated BMI, generated workout plans, nutrition tips and feedback history.
- Add user authentication and per-user plan history.
- Deploy the backend and frontend on a cloud platform for public access.
- Add progress tracking and visual reports of the user's fitness metrics.
- Improve the deterministic fallback with different structures per goal and intensity.
- Add automated tests for the backend endpoints and generation functions.
- Cache generated plans to reduce the number of Gemini requests and rate-limit errors.

Skill Wallet / Naan Mudhalvan Skills:

Project Title	FitBuddy AI – Intelligent Full-Stack Adaptive Fitness & Nutrition Coach
Student Name	Srihari
Team Member	Sanjay
Technologies	Python, FastAPI, Uvicorn, Streamlit, SQLite, SQLAlchemy, Google Gemini, Git, GitHub

Skills Demonstrated

No.	Skill	Where it is demonstrated in the project
1	Generative AI	Gemini-based workout, nutrition/recovery and adaptive plan generation
2	Python Programming	Backend logic, generation modules and Streamlit frontend
3	FastAPI	REST backend with GET /, POST /predictions and GET /predictions, served by Uvicorn
4	Streamlit	Interactive frontend at http://localhost:8501
5	SQLite	Lightweight file-based database (nexora.db)
6	SQLAlchemy	Engine, SessionLocal, Base and the Prediction model
7	REST API Integration	Frontend-backend HTTP communication and Gemini access through REST transport
8	Prompt Engineering	Prompts built from goal, intensity, user metrics and feedback
9	AI Application Development	Complete AI workflow from user input to stored, displayed result
10	Full-Stack Development	Frontend, backend, database and AI layer in one application
11	Fault Tolerance	Deterministic fallback keeps the application working when Gemini cannot respond
12	Retry and Backoff Handling	Retry logic with backoff delays for HTTP 429 rate-limit errors
13	Database Persistence	Generated results saved to SQLite through SQLAlchemy
14	Software Testing	End-to-end verification of the Streamlit-FastAPI-Gemini/fallback-SQLite pipeline
15	Git	Version control for the project
16	GitHub	Repository synchronized with GitHub; main branch up to date
17	Frontend-Backend Integration	Decoupled Streamlit and FastAPI integration with flexible parameter handling

Upload-ready Project Description

FitBuddy AI is a full-stack Generative AI fitness and nutrition application developed using Python, FastAPI, Streamlit, SQLite, SQLAlchemy and Google Gemini. The system generates personalized seven-day workout plans and nutrition/recovery guidance from user fitness inputs, supports feedback-based plan refinement, stores generated results, and includes retry and deterministic fallback mechanisms for AI-service rate limits.

Skills (comma-separated, for portal upload)

Generative AI, Python Programming, FastAPI, Streamlit, SQLite, SQLAlchemy, REST API Integration, Prompt Engineering, AI Application Development, Full-Stack Development, Fault Tolerance, Retry and Backoff Handling, Database Persistence, Software Testing, Git, GitHub, Frontend-Backend Integration

Conclusion:

FitBuddy AI – Intelligent Full-Stack Adaptive Fitness & Nutrition Coach, developed by Srihari and Sanjay, is a full-stack AI application that combines a Streamlit frontend, a FastAPI backend, a SQLite database accessed through SQLAlchemy, and Google Gemini integrated through the Google Generative AI interface. From user inputs such as name, age, weight, fitness goal and workout intensity, the system generates a personalized 7-day workout plan together with nutrition and recovery guidance.

The project goes beyond a single AI prompt. Feedback-based adaptive planning allows an updated plan to be produced as the user's needs change, while function aliases and flexible keyword-argument handling resolved the integration problems between modules. To deal with the limits of the Gemini free tier, the application uses retry logic with backoff delays and handles HTTP 429 rate-limit errors. If the AI request still cannot be completed, a deterministic fallback – a structured 7-day progressive workout split with a macronutrient-oriented protocol – keeps the application responsive. This fallback is a software reliability mechanism and not a medically validated prescription.

The complete pipeline – Streamlit input, FastAPI processing, Gemini or fallback generation, storage in SQLite and display in the frontend with a success indication – was verified end-to-end on Windows 11 with Python 3.10.11 in the fitbuddy-env environment. Development was managed with Git and GitHub, and the main branch is synchronized.

The decoupled architecture provides a strong base for future scalability, such as extending the database to store full user metrics, BMI and generated plans, adding authentication, and deploying to the cloud. Overall, the project gave the team practical experience in generative AI, REST API integration, database persistence, fault tolerance and full-stack application development.

Student: Srihari **Team Member:** Sanjay

References:

- 1 [FastAPI Documentation](#)
- 2 [Streamlit Documentation](#)
- 3 [Google Gemini API Documentation](#)
- 4 [SQLAlchemy Documentation](#)
- 5 [SQLite Documentation](#)
- 6 [Uvicorn Documentation](#)
- 7 [Python Documentation](#)
- 8 [Git Documentation](#)
- 9 [GitHub Docs](#)